

Alrazi University

Mr.PassKeeper: Hardware Password Manager

Semester: Summer 2026

Submission Date: September 29, 2026

Team Members:

- محمد فؤاد محمد اليوسفي_138
- عز الدين معين محمد عقيل_103
- منير عبدالله حمود احمد النجم_140
- محمد عبده محمد حاج مهيم_132
- هادي عبدالله ناصر صالح فطيح_149

Instructor:

Dr. Mahmoud E. Hodeish

Eng. Majd Alsurihi

2. Abstract

Mr.PassKeeper is an advanced hardware password manager designed on the ESP32 microcontroller. It combines an OLED display, BLE (Bluetooth Low Energy) keyboard emulation, and an isolated Wi-Fi web server to provide a secure, physical vault for password management. By isolating passwords from potentially compromised operating systems and eliminating cloud dependency, the system ensures maximum security, storing all data encrypted in the ESP32's NVS memory. With its plug-and-play capability and dynamic web dashboard, users can easily generate, store, and seamlessly type complex passwords into any BLE-supported device.

3.Introduction

3.1 Background and Motivation

As cyber threats, malware, and keyloggers become more sophisticated, software-based password managers are increasingly vulnerable. A physical, isolated device for storing passwords is highly motivated by the need for true air-gapped security, taking away the reliance on cloud storage. This project addresses the need for a low-cost, automated solution that bridges the gap between insecure software managers and expensive enterprise hardware vaults.

3.2 Problem Statement

Traditional software password managers are prone to OS vulnerabilities and keyloggers. Entering passwords manually is tedious, and cloud synchronization introduces the risk of mass data breaches. A system is required to securely store credentials locally and inject them into devices automatically as keystrokes.

3.3 Project Objectives

- Develop an ESP32-based hardware password manager.
- Emulate a Bluetooth Keyboard for cross-platform plug-and-play typing.
- Implement an intuitive local Web Dashboard for adding and managing credentials.
- Provide a hardware-based Random Password Generator.
- Display a UI on a 1.3" OLED screen with physical button navigation.

3.4 Scope and Limitations

The scope of this project is limited to a functional prototype. It is designed to work with any BLE-compatible host device. It does not feature cloud synchronization; if the device is damaged, data is lost unless manually backed up. Keyboard injection relies on the host device having an English keyboard layout active. It stores up to 100 accounts.

4.System Overview

4.1 High-Level Description

The system operates as a self-contained vault. The Central Processing Unit (ESP32) manages the system. When interacting with the web server via the ESP32's Wi-Fi Access Point, users can add or generate accounts. For usage, the user navigates the OLED UI using physical buttons to select an account, which triggers the BLE module to send the saved credentials as emulated keystrokes to the connected device.

4.2 Functional Description

- **Sensing / Input:** Hardware push buttons trigger UI navigation and injection events.
- **Processing:** The ESP32 executes C++ OOP logic to manage data, interface with the OLED, and control Wi-Fi/BLE stacks. Data is saved in NVS memory.
- **Visualization:** An I2C-enabled SH1106 OLED provides a user-friendly interface showing menus and connection statuses.
- **Alerting / Output:** BLE Keyboard emulator sends automated keystrokes to target devices.

4.3 Use-Case Scenario

Scenario: Logging into a Website The system is powered on, showing the "Mr.PassKeeper" logo. The user navigates the OLED menu to "Accounts", selects "GitHub", and chooses to send "User&Pass". The ESP32 automatically types the username, presses 'Tab', types the complex password, and presses 'Enter', logging the user in instantly without revealing the password on the PC's screen.

5. Hardware Design

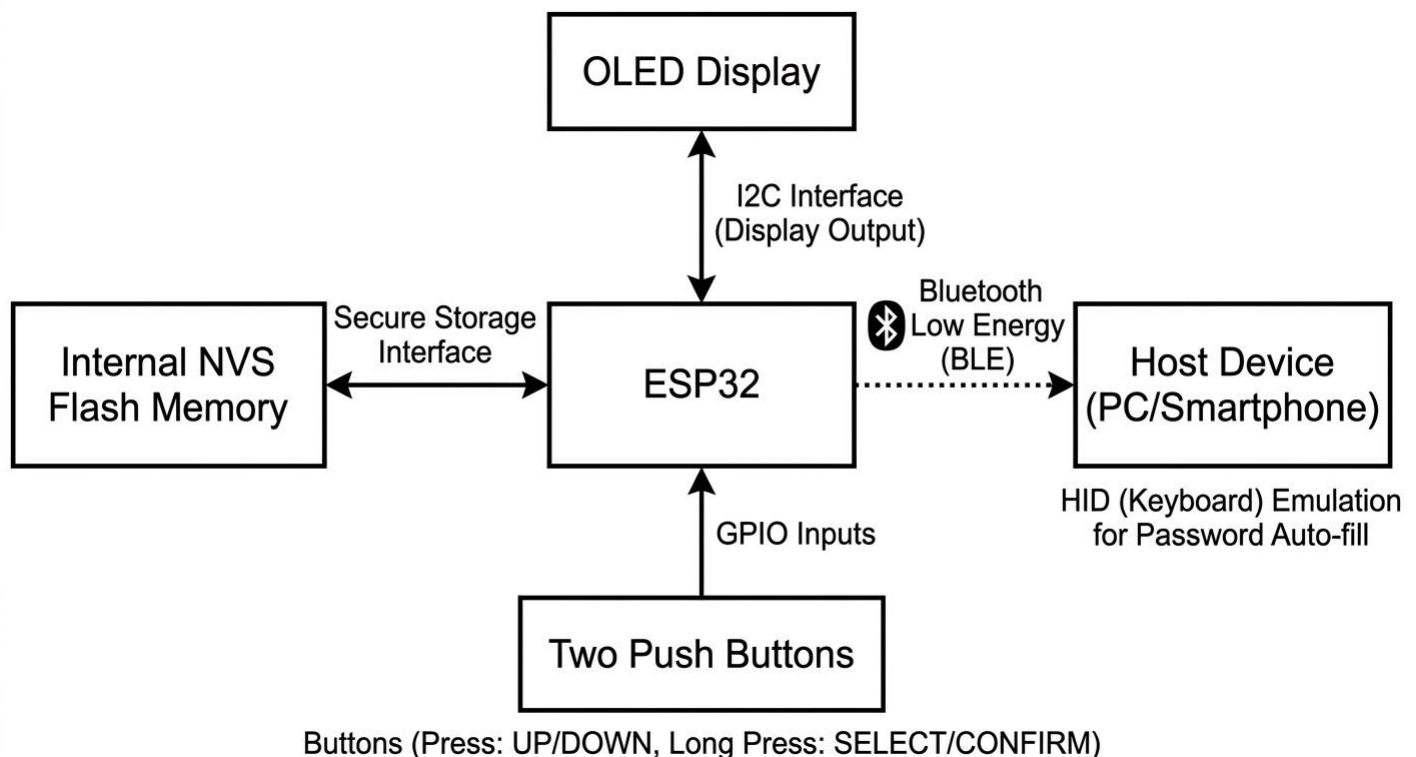
5.1 Components List

Component Name	Specification / Model	Quantity
Microcontroller	ESP32 (DOIT DEVKIT V1)	1
Display	1.3" OLED SH1106 (I2C)	1
Input	Push Buttons	3

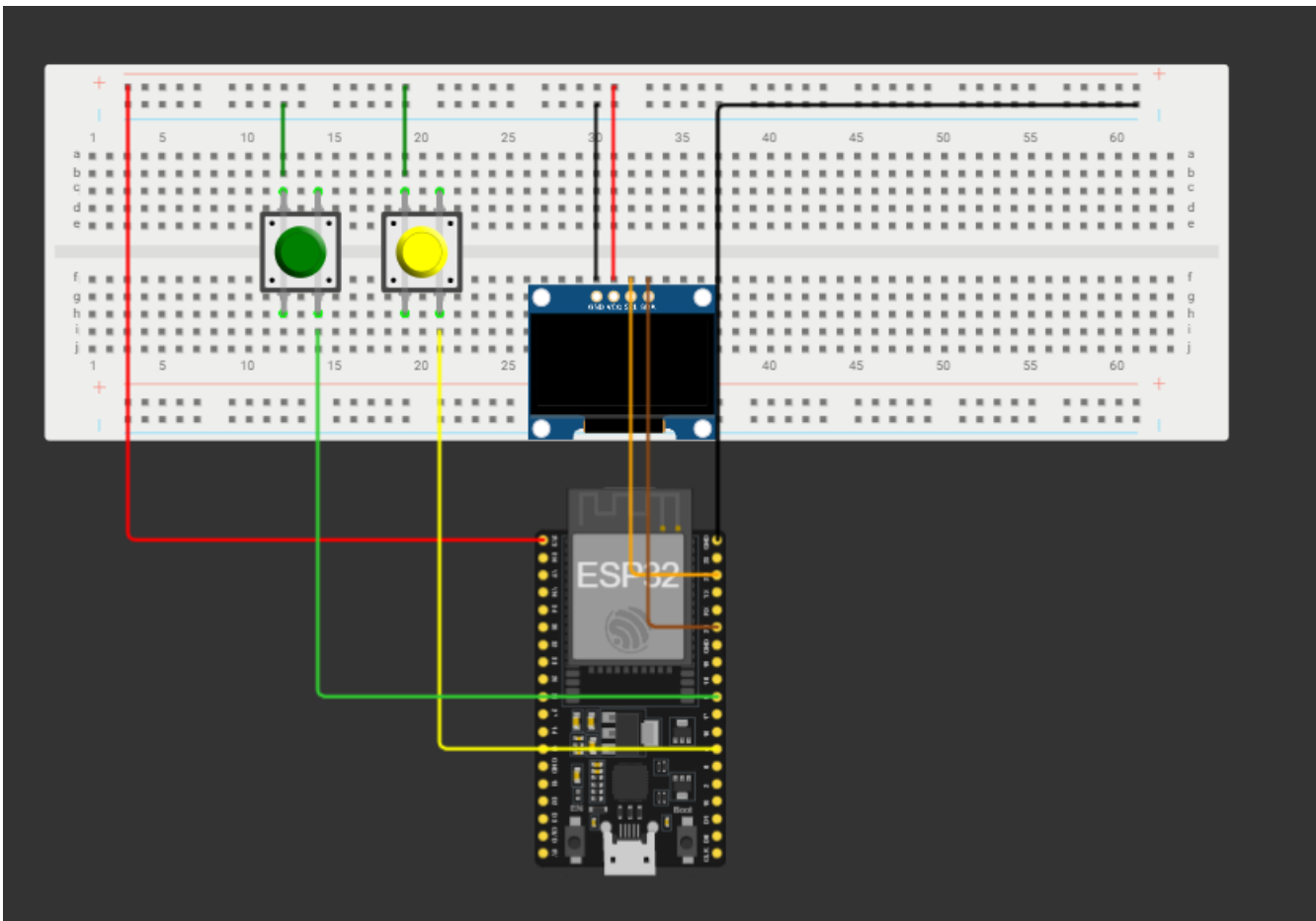
5.2 Component Justification

The **ESP32** was selected for its integrated Wi-Fi (for the Web Dashboard) and BLE (for keyboard emulation) capabilities, plus its adequate flash memory for the Web Server. The **SH1106 OLED** offers a crisp interface over just two I2C pins, saving GPIO. Hardware buttons provide reliable and isolated control without exposing interfaces to the network.

5.3 Block Diagram



5.4 Circuit Diagram



5.5 Power Considerations

The system is powered via the ESP32 USB port (5V DC) or battery module. The total current consumption is estimated at ~150-250mA during Wi-Fi operation and significantly less during BLE mode, well within standard USB limits.

6. Software Design

6.1 Development Environment

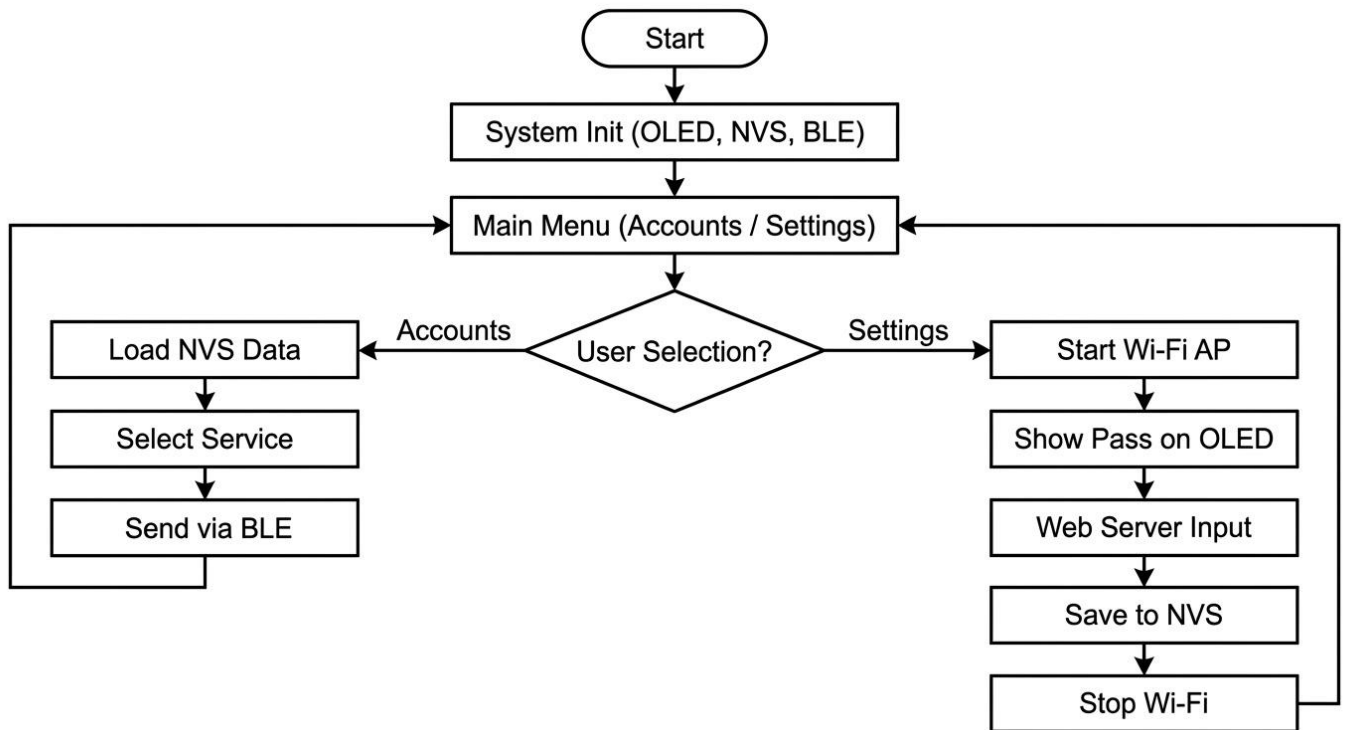
- **Language:** C++ (Embedded)
- **IDE:** PlatformIO
- **Libraries:** Adafruit_SH110X, Adafruit_GFX, BleKeyboard, WiFi, WebServer, Preferences

6.2 Software Architecture

The software follows an Object-Oriented Programming (OOP) architecture. `setup()` initializes I2C, Display, BLE, and the `clsMainScreen`. The `loop()` continuously runs `MainScreen.Update()` which handles state machines for screen navigation, button debouncing, and conditionally launching the Web Dashboard via Captive Portal.



6.3 Flowchart / State Diagram



6.4 Algorithms

```
if (bleKeyboard.isConnected()) {
  // Inject Username
  bleKeyboard.print(username);
  delay(30);

  // Switch to Password Field
  bleKeyboard.write(KEY_TAB);
  delay(30);

  // Inject Password and Submit
  bleKeyboard.print(password);
  delay(30);
  bleKeyboard.write(KEY_RETURN);
}
```

7.System Integration

PlatformIO configuration was modified (`board_build.partitions = huge_app.csv`) to accommodate both the BLE and Wi-Fi stacks concurrently, as they require significant flash space. The OLED utilizes the I2C protocol communicating on address `0x3C`. A 30ms delay was added to the `BleKeyboard` object to prevent dropped characters during rapid Bluetooth transmission.

8. Testing and Validation

8.1 Testing Strategy

We employed a bottom-up testing strategy. First, individual modules (Display, BLE, NVS Storage) were tested with simple sketches. Next, the Web Server was tested for memory leaks. Finally, full integration was tested across Windows, Android, and iOS devices.

8.2 Test Cases

Provide specific test cases with expected vs. actual outcomes.

Test Case ID	Input / Condition	Expected Output	Actual Output	Status (Pass/Fail)
TC-01	Enter Settings Mode	Wi-Fi AP starts, OLED shows Account	AP started, OLED updated	Pass
TC-02	Web Dashboard Save	Credentials saved to NVS memory	Data persisted after reboot	Pass
TC-03	BLE Injection (User&Pass)	Correct characters typed with Tab/Enter	Typed perfectly on Windows	Pass
TC-04	High Speed Typing	No dropped characters	30ms delay ensured no drops	Pass

8.3 Hardware and Software Testing

Software debugging relied on the Serial Monitor to verify NVS read/write operations and Web Server POST requests. Hardware testing verified that button presses were correctly debounced in software to prevent double-injections.

9. Results and Discussion

The password manager functions reliably within design parameters. BLE connection is established in under 2 seconds, and typing a 16-character password takes less than a second. The dynamic OOP architecture allows for infinite accounts to be added (limited only by NVS). CPU and memory consumption are highly optimized (18% RAM, 52% Flash).

10. Challenges and Solutions

- **Challenge:** Characters dropping when typing fast over BLE.
- **Solution:** Implemented `bleKeyboard.setDelay(30)` to introduce a micro-pause between keystrokes, fixing the issue.
- **Challenge:** Code size exceeded the default ESP32 partition limits due to large Wi-Fi and BLE libraries.
- **Solution:** Modified `platformio.ini` to use the `huge_app.csv` partition scheme.
- **Challenge:** Wrong characters typed if host PC keyboard is Arabic.
- **Solution:** Documented limitation; host keyboard must be set to English.

11. Conclusion

The "Mr.PassKeeper" project successfully met all objectives. It provides a highly secure, air-gapped solution to password management that seamlessly integrates with modern devices via BLE. The integration of a local web dashboard for management removes the need for custom apps, ensuring universal compatibility and a solid foundation for future enhancements.

12. Future Work

- **Security:** Integrate a biometric fingerprint sensor module to authenticate and unlock the device upon startup.
- **Data Backup:** Integrate an SD Card module to allow encrypted backups and restoration of the password vault.
- **Localization:** Develop algorithms to support sending ALT-codes to bypass keyboard language layout limitations.

13. References

1. Espressif Systems. ESP32 Documentation.
2. Adafruit Industries. SH110X & GFX Library.
3. T-vK. ESP32-BLE-Keyboard Library.

14. Appendices

Appendix A: Source Code

The complete source code for this project is hosted on our team repository.
<https://github.com/ProMr-SiR/PassKeeper>

Appendix B: Additional Schematics/Figures

Attached below are the pinout diagrams for the ESP32 DEVKIT V1 and the SH1106 OLED module used in this project.

